Skip to main content

ILLINOIS

Search in course

Search

- English
-
- Menu pop-up Collapsed

B

Hide menu

- **Week 6 Introduction**
- **Relational Databases on the Cloud**
- **NoSQL Databases on the Cloud**
- **NewSQL Cloud Databases**
- **Week 6 Graded Activities**
  -

Graded Assignment: Week 6 Quiz

. Duration: 1 hour1h

  -

Programming Assignment: Machine Problem 6 V2: Aurora and ElastiCache

. Duration: 5 hours5h

1. CS 498: Cloud Computing Applications

2. Week 6
3. Machine Problem 6 V2: Aurora and ElastiCache

PreviousNext

Programming Assignment: Machine Problem 6 V2: Aurora and ElastiCache

Graded • 4/4 points

Deadline

Pass this assignment by Mar 30, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

This MP explores AWS RDS (Relational Database Service) and ElastiCache services. Specifically, you will learn how to use ElastiCache and RDS from Lambda and see how using cache can reduce query response times. You will build a small storage service in Lambda to support read and write queries on data stored in a relational database. Your goal is to demonstrate the performance benefits of using caching to improve the performance of database queries.

2. Requirements

You need to use the following AWS services for this MP - S3, Lambda, RDS, Elasticache, API Gateway, IAM and EC2 (optional). You need a valid AWS account to work on these services.

Note: AWS Educate accounts do not have access to Redis Service, so you will need to use your personal account in this MP. We suggest you to use a single personal AWS account for the entire MP. Except RDS all other services fall under free tier as of 2023. You can check the exact Free tier resources and limits here.

Also, you need to be familiar with one of the following programming languages for implementing Lambda: Python / Javascript / Java / Go. While we will attempt to help out irrespective of your chosen language, we can best assist with python 3.

Note: Please run all your services in us-east-1.

Note: Delete all the resources after finishing the MP to save charges.

Note: All the resources you create must use same VPC and subnet groups. Use security groups for resources whenever required to allow appropriate traffic.

3. Create a Relational Database

First, you need to create a Relational Database in RDS and populate it with the data we provide.

3.1 Amazon S3 setup

Create an S3 bucket and upload the following file to the bucket:

mp6data.csv (download the file)

Getting started with Amazon S3

3.2 Setup and populate RDS database

Step 1: Create an IAM policy and IAM role for Amazon RDS to access Amazon S3

Create an IAM policy for S3 to access S3 files. Next, create an IAM role for RDS and attach the previously created S3 policy. You also need to create an RDS cluster parameter group and set the S3 parameters with the RDS IAM Role. You will need this cluster parameter group later while creating the database.

Follow these tutorials to set up the required permissions:

Creating an IAM policy to access Amazon S3 resources

Creating an IAM role to allow Amazon Aurora to access AWS services

Associating an IAM role with an Amazon Aurora MySQL DB cluster

Step 2: Create an RDS database

Create an Amazon Aurora RDS database. Refer the two tutorials listed below for instructions. Make sure you use standard create and use "DB instance class": Burstable classes (includes t classes). This configuration is to ensure you use the small database to avoid large charges ($$$). Ensure "publicly accessible" is selected. While creating the database, use the cluster parameter group you created in the above step. Attach the RDS IAM Role you created to the database. Please ensure that you use the smallest possible instance of RDS.

Creating and connecting to an Aurora MySQL DB cluster (You can connect to the database from your EC2 instance or any other way like using MySQL workbench. Next step has some details about this).

Associating an IAM role with an Amazon Aurora MySQL DB cluster

Step 3: Populate database using data in Amazon S3

Load the data stored in the S3 bucket into the database. Note that the csv file in S3 has header as well. Refer the tutorial:

Loading data into an Amazon Aurora MySQL DB cluster from text files in an Amazon S3 bucket

To access the database, you can use MySQL from an EC2 instance described in this tutorial.

After setting up your instance, you can connect to the database from your instance with the following command: " **mysql -h rds_writer_endpoint -P 3306 -u admin -p"**

Before loading the csv file into Aurora, you should create a database and a table. The table should have the following fields:

id : INT unsigned (Primary Key)

name : VARCHAR

hero : VARCHAR

power : VARCHAR

xp: INT unsigned

color : VARCHAR

Don't change these field names and type as as the autograder is hardcoded with these names. Also, make sure you set length/size of each VARCHAR column to **20** because the autograder inserts randomly generated data into the table for testing.

For basic MySQL operations, this tutorial should help.

If you face issues with loading data from s3 or you are seeing the 'Unable to initialize S3Stream' error, please refer to the following article and ensure that all steps are covered. https://rspacesamuel.medium.com/setting-up-amazon-aurora-to-read-from-s3-e90661ca57f0

4. Create a Redis Cluster

Create a Elasticache Redis OSS cluster. Refer this tutorial for the instructions. The cluster should be created with the cluster mode disabled. This is a good resource to understand how the Elasticache cluster mode works.

Note: AWS Educate accounts do not support Redis, you will require a personal account to create this cluster.

After creating the cluster, you can test the connection from your EC2 instance (or local computer). You can refer to this tutorial.

5. Create a Lambda function

Create a Lambda function to access your RDS database and Redis cluster. You need to implement the logic to support database reads and writes.

5.1 Setting up Lambda function

We provided a template for the Lambda. You need to write the code and complete parts marked "# TODO". Note that this template is for Python. More instructions about the read and write functionalities are given below.

You can refer to MP3 for creating Lambda functions. Unlike some previous MPs, you can import third-party libraries in this MP. If you choose to use Python for this MP, you will need to refer to this tutorial for importing two modules pymysql and redis. You can also refer to this tutorial.

If you decide to use other languages, please refer to the AWS official documentation to figure out how to import third-party libraries.

The following example might also help you - amazon-elasticache-samples/database-caching

Note: Upon creation, Lambda will not be in any VPC. But, for this MP to work, configure the Lambda to be in the same VPC and subnet groups as Redis and RDS.

5.2 Read and Write API

There are different strategies for using caches. Here, we utilize **write-through** and **lazy-loading** strategies. You can refer to this document for more details.

Useful resource: Boosting MySQL database performance

There will be 2 types of requests which your lambda should support - **"Read"** and **"Write".**

Read:

The autograder will send read requests in the following format:

1

2

3

```
4
5
{
  "USE_CACHE": "True",
  "REQUEST": "read",
  "SQLS": [1,2,3]
}
```

You must read and return all the rows with the given **xp** (**xp**s given in the "SQLS" parameter) in the **same order** as the order of **xp**s in the "SQLS" parameter. If USE_CACHE is true, check if Redis already contains the data with the given **xp**. If redis doesn't contain the data, read the row from RDS, store it in Redis, and return the result. If USE_CACHE is false, directly read from RDS database. Note that the choice of key and value in Redis is totally up to you. The autograder will **only** ask you for the corresponding rows of the **xp**s.

Note that the query here is not on "**id**" but it is on "**xp**" column. Also note that the **xp** column is unique. All the rows in the database will have a different **xp** value. But this field is not a Primary Key for the table.

At the end of your lambda handler function, the return statement should be as follows:

```
1
2
3
4
return {
   "statusCode": 200,
   "body": result
}
```

Result format : each row of data is represented as a dictionary. You should put all the result rows in a list. Therefore, you will need to return a **list of dictionaries** as string format. Finally, the result should be in **JSON** format. An example is shown below.

```
1
```

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

```
28
29
return {
 "statusCode": 200,
 "body": [
  {
   "id": 1000000,
   "name": "antdoctor",
   "hero": "no",
   "power": "fly",
   "xp": 1,
   "color": "red"
  },
  {
   "id": 999999,
   "name": "windwoman",
   "hero": "yes",
   "power": "fly",
   "xp": 2,
   "color": "yellow"
  },
  {
   "id": 999998,
   "name": "shieldman",
   "hero": "no",
   "power": "scream",
```

```
    "xp": 3,
    "color": "grey"
  }
 ]
}
```


Write:

For a write request, you must insert new superhero characters in the Aurora database . An example of a write request API call is as follows:

```
19
20
{
 "USE_CACHE": "True",
 "REQUEST": "write",
 "SQLS": [
  {
   "hero": "yes",
   "name": "fireman",
   "power": "fire",
   "color": "red",
   "xp": "10"
  },
  {
   "hero": "no",
   "name": "dogman",
   "power": "bark",
   "color": "brown",
   "xp": "50"
  }
 ]
}
```

Insert the rows to the table according to the order given in the"SQLS" parameter. Note that "**id**" is not provided in the request. You need to increment the id and insert. Functionality for this is already provided in the template.

If USE_CACHE is True, you should use a write-through strategy. Our autograder will check whether your write-through implementation improves the read performance. If

USE_CACHE is False, do not use any cache. For write API calls, the result body should be **"write success".**

The write requests we send from the autograder will make sure that the **xp** value is unique and does not exist in the table already.

5.3 Testing your Lambda

Once you are done with the Lambda, you can test it by using Lambda test events. Use the sample read and write requests above to test the Lambda. You can check the logs in Cloudwatch.

6. API setup

Set up an API endpoint for your Lambda function. Create a REST API in the API Gateway and integrate with the Lambda.

Important note:

After you create and test your lambda code, you can integrate it with API Gateway primarily in 2 ways:

- Adding a trigger directly from the lambda console using the "Add trigger" function.
- Creating an API from API Gateway console and integrating with lambda.

In the first method, the API will be created with Lambda Proxy integration which shouldn't be used. Lambda Proxy integration is different from Lambda integration. The functionality, request and response models for proxy integration are different than Lambda integration. The sample request models, response models and the lambda template provided above do not work when you use Lambda Proxy integration. The autograder may also face issues if you use proxy integration. You may get this error in such scenarios -

"Unable to jsonify response or no 'body' key in json"

Therefore, you can create an API from the API gateway console and make sure you DO NOT check the box "Use Lambda Proxy integration" while creating the POST method. If you create the API from Lambda console, then you need to create a new POST method (in the api gateway console) without Lambda proxy integration.

If you want to learn more, refer Using AWS Lambda with Amazon API Gateway, Understand API Gateway Lambda proxy integration

7. Autograder

Our autograder runs 4 tests on your Lambda:

Test 1: Read without cache - We make Read requests to your Lambda with USE_CACHE set to False. We check whether the correct rows are returned

Test 2: Read with cache - We make the same Read requests to your Lambda as in Test 1, but with USE_CACHE set to True. We check whether you returned correct rows or not. Also, we check whether the response time for the queries in this test is lesser than the time taken in Test 1. If time for Test 2 is more than Test 1, then you will receive an error and this test fails.

Test 3: Write without cache - We make Write requests with USE_CACHE as False. We then make a read request to read all the rows you have written and check whether they are correct or not.

Test 4: Write with cache - We make Write requests with USE_CACHE as True. We then make a read request to read all the rows you have written and check whether they are correct or not. We also check whether the response time in Test 4 is lesser than time in Test 3. If time for Test 4 is more than Test 3 , then you will receive an error and this test fails.

Each test is worth 1 point. The Autograder should typically finish running in approximately 18 seconds if the solution is implemented correctly in an optimal way.

You can print the response times in your Lambda and check them while testing the Lambda even before making a submission.

8. Submission

Download the following file. Fill in your email, assignment token and the API endpoint, and then run it on your machine.

submission.py

Note: Before making every submission, do the following:

1. Delete all rows in the table with id>1000000 - ""DELETE FROM {table_name} WHERE id > 1000000"

2. Wait for at least 1 minute between each submission.

8. Important Notes

1. Make sure you "DELETE FROM {table_name} WHERE id >= 1000000" every time you make a submission.
2. The communication overhead sometimes outweighs the performance boost. If you are sure your implementation of write-through and lazy-loading strategy is correct

but test 2 or test 4 fails due to no performance boost, then you can reboot your redis nodes and

3. If you get an "unable to jsonify response or no 'body' key in json" error response from the autograder, you can try replicating the problem using the following code. Copy this code in a new file, replace "api" with your POST method URL and run it.

```
payload = {
    "USE_CACHE": "True",
    "REQUEST": "read",
    "SQLS": [1,2,3]
  }
api = "your dbApi"
r = requests.post(api, data=json.dumps(payload))
res = ""
res = r.json()['body']
```

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

Your token expired

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue